Análisis y Optimización de Algoritmos de Ordenamiento: Quicksort de 3 Vías (Quick3) en Conjuntos con Pocas Llaves Únicas

Javier Santiago Páez Penagos
Ingeniería de Software
Universidad de La Salle

Resumen

El algoritmo de ordenamiento Quicksort es ampliamente utilizado por su eficiencia promedio de $O(n \log n)$. Sin embargo, su implementación tradicional presenta una degradación severa de rendimiento al enfrentarse a conjuntos de datos con una alta repetición de elementos, conocidos como escenarios de "pocas llaves únicas" (Few Unique Keys), cayendo a una complejidad de $O(n^2)$. Este artículo presenta y analiza el algoritmo Quicksort de 3 Vías (Quick3), propuesto por Bentley y McIlroy, el cual implementa una partición tricotómica que agrupa los elementos duplicados y los excluye de las llamadas recursivas subsiguientes. Se demuestra cómo esta optimización reduce la complejidad temporal a $O(n)$ en presencia de duplicados masivos, ilustrando su aplicación práctica en un sistema logístico de clasificación.

***Palabras Clave*— Algoritmos de Ordenamiento, Bentley-McIlroy, Complejidad Algorítmica, Pocas Llaves Únicas, Quick3, Quicksort.**

1 Introducción

En el ámbito de las ciencias de la computación, la ordenación de datos es un problema fundamental con impacto directo en el rendimiento de bases de datos, motores de búsqueda y sistemas de gestión de información. Quicksort, desarrollado originalmente por Tony Hoare, es el algoritmo de ordenamiento de propósito general por excelencia debido a su alta velocidad y su bajo consumo de memoria (al ordenar in-place) [2].

A pesar de su eficiencia, la implementación estándar de Quicksort se basa en una partición de dos vías que divide el arreglo en elementos menores o iguales al pivote, y elementos mayores al pivote. Este enfoque asume implícitamente que la mayoría de los elementos del conjunto son únicos. Cuando se aplica a conjuntos de datos donde existen pocas llaves únicas (es decir, pocos valores distintos repetidos múltiples veces), el algoritmo realiza comparaciones e intercambios redundantes que degradan su rendimiento hasta una complejidad cuadrática $O(n^2)$ [3].

Este artículo explora la solución a esta vulnerabilidad: el Quicksort de 3 Vías, también conocido como partición de Bentley-McIlroy. Se analizará su mecánica interna, su análisis de complejidad asintótica y se expondrá un caso práctico que demuestra su viabilidad en aplicaciones de ingeniería de software.

2 El Problema del Quicksort Tradicional

El comportamiento de Quicksort depende críticamente de la calidad de la partición. En su forma clásica, tras seleccionar un pivote, los punteros recorren el arreglo intercambiando elementos para asegurar que todos los valores menores queden a la izquierda y los mayores a la derecha.

Si un arreglo contiene un alto porcentaje de elementos idénticos al pivote, el algoritmo estándar no los reconoce como ya ordenados. En su lugar, los distribuye arbitrariamente en las sub-listas izquierda o derecha. Esto provoca que en las siguientes llamadas recursivas, el algoritmo vuelva a comparar y mover estos elementos. Si un arreglo consta únicamente de elementos idénticos, la partición estándar solo reducirá el tamaño del problema en un elemento por cada llamada recursiva, resultando en la relación de recurrencia:

$$T(n) = T(n-1) + O(n) \tag{1}$$

La solución a esta recurrencia es $O(n^2)$, lo cual es inaceptable para sistemas a gran escala [1].

3 Quicksort de 3 Vías (Partición Tricotómica)

Para solucionar la ineficiencia ante duplicados, Jon Bentley y Douglas McIlroy propusieron en 1993 una mejora fundamental: dividir el arreglo en tres regiones en lugar de dos [1].

3.1 Mecánica de los Punteros

El algoritmo Quick3 utiliza tres punteros principales (lt, i, gt) durante el proceso de partición de un subarreglo delimitado por low y $high$. Al elegir un pivote $v = arr[low]$, los punteros mantienen el siguiente invariante a medida que i recorre el arreglo de izquierda a derecha:

- $arr[low \dots lt-1]$: Elementos estrictamente menores que v.
- $arr[lt \dots i-1]$: Elementos iguales a v.
- $arr[i \dots gt]$: Elementos aún no procesados.
- $arr[gt+1 \dots high]$: Elementos estrictamente mayores que v.

Durante el escaneo del elemento actual $arr[i]$, pueden ocurrir tres casos:

1. **Menor que el pivote**: Se intercambia $arr[i]$ con $arr[lt]$, y se incrementan tanto i como lt.
2. **Mayor que el pivote**: Se intercambia $arr[i]$ con $arr[gt]$, y se decrementa gt. El puntero i no avanza, ya que el elemento traído desde la derecha debe ser evaluado.
3. **Igual al pivote**: Simplemente se incrementa el puntero i.

3.2 Optimización de la Recursión

Una vez que el puntero de inspección i cruza el límite de gt, el ciclo termina. El arreglo queda particionado y todos los elementos iguales al pivote se encuentran correctamente ubicados en el centro (entre lt y gt).

El aspecto más importante de esta optimización es que las llamadas recursivas se aplican exclusivamente a las regiones laterales:

```
quicksort(arr, low, lt - 1)
quicksort(arr, gt + 1, high)
```

La sub-lista central $[lt \dots gt]$ queda completamente excluida de procesamientos futuros, ahorrando drásticamente tiempo computacional y espacio de pila.

4 Análisis de Complejidad

El rendimiento asintótico del Quicksort de 3 Vías varía según la entropía de los datos (la cantidad de llaves únicas).

4.1 Peor Caso y Caso Promedio

Si todos los elementos del arreglo son distintos, el algoritmo se comporta de manera similar al Quicksort tradicional. La complejidad promedio sigue siendo:

$$O(n \log n) \tag{2}$$

4.2 Mejor Caso (Pocas Llaves Únicas)

Cuando el número de llaves únicas es muy bajo o constante independientemente de n

(Few Unique Keys), la partición agrupa bloques inmensos de elementos duplicados en la zona central. En el caso extremo donde todos los elementos son iguales, el ciclo de partición los procesa una sola vez, y la recursión se detiene de inmediato. En estos escenarios, el tiempo de ejecución se reduce a:

$$O(n) \tag{3}$$

Esta característica hace que Quick3 sea un algoritmo *adaptativo*, mejorando su velocidad a medida que el arreglo presenta más elementos duplicados.

5 Caso de Estudio: Aplicación Logística

Para validar la eficiencia técnica del modelo, se conceptualizó un sistema de gestión logística para un almacén de distribución.

5.1 Descripción del Entorno

Se generó computacionalmente un lote de 50.000 registros de órdenes de compra. Dado el dominio del negocio, cada orden solo podía tener una de tres clasificaciones de prioridad:

- 1: Prioridad Urgente.
- 2: Prioridad Normal.
- 3: Prioridad Programada.

Esto constituye un escenario perfecto de "Pocas llaves únicas".

5.2 Resultados

Al ejecutar el Quicksort estándar, el algoritmo generó miles de intercambios innecesarios entre paquetes con la misma prioridad. Por el contrario, la implementación de Quick3 logró organizar los 50.000 paquetes en fracciones de milisegundo. En una sola pasada, los paquetes de prioridad '2' quedaron agrupados en el centro, obligando al sistema a ordenar recursivamente solo las regiones restantes, confirmando la complejidad teórica lineal empíricamente.

6 Conclusiones

El algoritmo Quicksort de 3 vías (Quick3) soluciona de forma elegante y contundente la vulnerabilidad de las llaves duplicadas del Quicksort clásico. La división tricotómica garantiza que los elementos iguales al pivote alcancen su posición final definitiva de inmediato. Esta optimización es tan significativa para el rendimiento en escenarios del mundo real que lenguajes de programación modernos y bibliotecas estándar utilizan variaciones del Bentley-McIlroy como motor principal de sus funciones de ordenamiento nativas.

Referencias

[1] J. Bentley y D. McIlroy, "Engineering a Sort Function," *Software—Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.

[2] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.

[3] R. Sedgewick y K. Wayne, *Algorithms*, 4th ed., Addison-Wesley Professional, 2011, pp. 288–296.